

Code Assessment of the LZ Init Lib Smart Contracts

September 07, 2026

Produced for



Contents

1	Executive Summary	3
2	Assessment Overview	4
3	System Overview	5
4	Trust Model	9
5	System Considerations	10
6	Terminology	12
7	Findings	13
8	Limitations and use of report	16

1 Executive Summary

Dear all,

Thank you for trusting us to help Sky with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of LZ Init Lib according to [Scope](#) to support you in forming an opinion on their security risks.

Sky implements a Solidity library for configuring LayerZero v2 infrastructure (peer wiring, message-library selection, DVN and executor configuration, OFT rate limits) from within Sky governance spells, and a companion helper contract that exposes the same library on L2 via the Sky cross-chain governance relay. In [Version 3](#), a one-off library for the Avalanche OFT and governance migration was added. In [Version 4](#), support for activating SSR oracle forwarders was added, and the use of the shared CCIP DVN adapter was made optional when wiring a new governance peer.

The subjects covered in our audit are functional correctness, completeness of operations and documentation.

In summary, we find that the codebase provides a good level of security. Note that the library configures contracts which are deployed and pre-configured outside of governance, and that its on-chain checks can confirm only that expected values are present, not that unexpected ones are absent. The correctness of those deployments therefore rests on Sky's off-chain review, see [Deployment verification](#).

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,
ChainSecurity

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The assessment was performed on the source code files inside the LZ Init Lib repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

Version	Date	Commit Hash	Note
1	11 May 2026	81ba8961fb90cccd5f957ccf54f44d5bb23e5b2a	Initial Review
2	21 May 2026	e2f900fa60ed5759764e8838d40d478de9a5f861	After Intermediate Report
3	11 August 2026	2bb0116c555bc354243df5704e5d38db47459c35	Avalanche Migration
4	19 August 2026	e717ef6c451859ea170aafaa4ddc44325ed0867c	SSR Oracle Forwarder
5	7 September 2026	70d6e028e8950a53f9de788f78b4d56ab77976b5	Fixes

For the Solidity smart contracts, the compiler version range `>=0.8.24 <0.9.0` is supported, derived from `pragma solidity >=0.8.0` in `LZInit.sol` and `LZAvaxMigrationInit.sol` and `pragma solidity ^0.8.24` in `LZL2Spell.sol` and `LZAvaxMigrationL2Spell.sol`. Since **Version 4**, `foundry.toml` pins the compiler to `0.8.24`; in earlier versions no compiler version was pinned and the supported range was `>=0.8.22 <0.9.0`. The EVM version is set to `cancun` in all versions.

```
deploy/
├─ LZInit.sol
└─ LZL2Spell.sol
```

In **Version 3**, the following files were added to the scope:

```
deploy/
├─ LZAvaxMigrationInit.sol
└─ LZAvaxMigrationL2Spell.sol
```

2.1.1 Excluded from scope

Any file not listed above is out of scope, including off-chain deployment scripts and pre-configuration of remote OApps (the responsibility of the deployer), the update of the Safe Harbor agreement with the new adapter and relay addresses (not performed by the library, as such updates use dedicated tooling and are handled directly at the spell level), the LayerZero v2 and CCIP protocol contracts (including the prefunding of the CCIP DVN adapter), and the Sky governance, OFT and DVN contracts; the latter have been covered by prior reviews.

In particular, the following contracts are not part of this assessment, even where this report describes how the library interacts with them: the contracts deployed ahead of the Avalanche migration, namely the new L1 lockboxes, the new Avalanche adapters and the new `L2GovernanceRelay` (see `Deployment verification`); the shared `CCIPDVNAdapter` and the multisig broadcaster and replica DVNs; and, as of **Version 4**, the SSR oracle forwarder, its remote receiver and the remote SSR oracle.

3 System Overview

This system overview describes the latest version of the contracts as defined in the [Assessment Overview](#).

Furthermore, in the findings section, we have added a version icon to each of the findings to increase the readability of the report.

Sky offers a library for configuring LayerZero v2 infrastructure from within Sky governance spells. It is intended to be used in two places:

- Directly as part of L1 governance spells, which import `LZInit` and invoke its functions in their own execution context.
- Indirectly on L2, through the cross-chain spell helper `LZL2Spell`, which is deployed once per L2 chain and is delegatecalled by the `L2GovernanceRelay` when a governance message arrives from L1. It exposes the L2-usable subset of `LZInit` so the same library code can be reached on remote chains, supporting wiring of L2 OApps to peers on any other chain, including other L2s. Note that it additionally exposes `multicall` to allow for batching.

Access control is expected to be configured prior to usage of the library. More specifically, `OApp.transferOwnership` and the LayerZero delegate assignment (`EndpointV2.setDelegate`, invoked via `OApp.setDelegate`) on every OApp touched are expected to have been performed by the deployer and validated by Sky governance for correctness beforehand.

Wiring an OApp to a remote peer is per-remote-EID and per-direction (send and/or receive), and consists of:

- Peer: `OApp.setPeer` records the counterparty address.
- Library: `EndpointV2.setSendLibrary` / `EndpointV2.setReceiveLibrary` binds the message library per direction per remote EID. (`EndpointV2.setReceiveLibraryTimeout` exists but is unused by this library since the grace period is set to and should be 0.) The library can be configured via `EndpointV2.setConfig`:
 - `ExecutorConfig` (type `1`, send only): destination executor and max message size.
 - `UlnConfig` (type `2`): DVN set and threshold as well as other related ULN settings.

For the governance OApp and the governance relay, the library exposes the following functionality which is only intended to be used on L1 and thus assumes that the receiving L2 governance OApp and relay have been configured correctly:

- `wireGovPeer`: Wires `LZ_GOV_SENDER`'s send side to a new remote receiver. Calls `setCanCallTarget` to whitelist `LZ_GOV_RELAY` for the remote relay. The counterparty on the receive side is expected to be configured on deployment. Since **Version 4**, the caller indicates via `GovConfig.ccipDvnIndex` whether the send DVN set includes the shared `CCIPDVNAdapter`. If it does, the index selects the adapter out of `sendUlnCfg.optionalDVNs`, which enforces its membership in the set, and the adapter's route to the new remote is asserted to be present. The sentinel value `NO_CCIP_DVN` indicates that the adapter is not used, in which case no assertion is performed. The route itself is expected to have been wired beforehand by `LZDVNInit.wireCCIPDVN` from the sister library `lz-gov-dvns-deploy`, which is not in scope.
- `relayToL2`: Dispatches a call to a spell on a remote chain via `L1GovernanceRelay.relayEVM`. Encodes the `lzReceive` option (gas), quotes the fee, and ensures bound native-only fees and that the L1 relay is pre-funded sufficiently. Note that `LZL2Spell` is a potential L2 spell but not the only one.

For OFTs, the library exposes the following functionality for L1 and L2:

- `wireOftPeer`: Used to add wiring to existing OFTs. Wires the local OFT for sending and receiving to and from the remote OFT. Additionally, pins `lzReceive` enforced options for `MSG_TYPE_SEND` and `MSG_TYPE_SEND_AND_CALL` (gas), and seeds rate limits via `updateRateLimits`. Fresh-wire only.

- `activateOft` : Used when activating an OFT that had no rate limits on deployment. Sets the rate limits for an otherwise fully configured deployment. Verifies the configurations and flips rate limits non-zero via `updateRateLimits`. Since **Version 4**, the expected LayerZero endpoint is passed in by the caller and checked against the OFT's own endpoint, instead of being read from the OFT.
- `updateRateLimits` : Sets one inbound and one outbound entry for `remoteEid` via `OFTAdapter.setRateLimits`.
- `updateGlobalRateLimits` : Sets the global (`SENTINEL_EID`) inbound and outbound entry via `updateRateLimits`, i.e. the L1 lockbox's aggregate cap across all remotes on top of the per-eid buckets. Sky OFT Adapter V2 only, V1 adapters do not support the new global rate limit.
- `unpauseOft` : Pass-through to `OFTAdapter.unpause`.

Sky OFT Adapter V2 differs only in adding upgradeability (UUPS proxy) and the global rate limit. All of the above therefore applies to both versions, except `updateGlobalRateLimits` and `activateOft`. No upgrade related functionality exists in the library yet.

For both, the Gov OApp and OFTs, the ULN config can be updated with (L1 & L2):

- `setUlnConfig` : Writes one `UlnConfig` entry against the supplied library; caller picks send or receive via the library address. Generic over any OApp (also usable for governance DVN-set migrations).

3.1 SSR Oracle Forwarder

In **Version 4**, the library gained support for onboarding an SSR oracle bridge to a new chain. Such a bridge consists of an L1 forwarder, a remote receiver and a remote SSR oracle.

- `activateSsrForwarder` : Verifies an SSR oracle forwarder's configuration on L1 and, when the forwarder uses the shared `CCIPDVNAdapter`, asserts the adapter's route to the destination and grants the forwarder the adapter's `ALLOWLIST` role. L1-only.

The `ALLOWLIST` grant is what activates the forwarder, so the function is an activation step rather than a pure configuration step. Granting the role requires the caller to hold the adapter's `DEFAULT_ADMIN_ROLE`, which is expected to be the pause proxy. As for `wireGovPeer`, the adapter is selected out of `sendUlnCfg.optionalDVNs` by index and `NO_CCIP_DVN` indicates that it is not used; in that case the function performs the configuration checks only and grants nothing.

The verification covers the forwarder's endpoint, destination EID, the sUSDS address it reports, its peer, its owner and delegate, its send library and that this library is not the chain default, the executor config, the send `UlnConfig` retrieved via `getAppUlnConfig`, and the enforced options for `MSG_TYPE_SEND`. The forwarder only ever sends, so no receive-side configuration is verified. The enforced options now pin an `lzCompose` component in addition to `lzReceive`, encoded by the library itself rather than through LayerZero's `OptionsBuilder`.

The remote receiver, the remote SSR oracle and the remote `CCIPDVNAdapter` configuration are not checked on-chain and are assumed to have been verified off-chain.

3.2 Avalanche Migration

`LZAvaxMigrationInit` is a one-off library that migrates the USDS and sUSDS OFTs to v2 OFTs, an upgradeable implementation that adds global rate-limit support (implemented in [PR #46 of sky-oapp-oft](#)). Alongside the token bridges, it reconfigures the LayerZero governance relay (implemented in [PR #2 of lz-governance-relay](#)), which adds timelock and guardian functionality. Because this is the first migration of the OFT and governance stack, the library additionally migrates the governance OApp to the intended 3-wing SkyLink infrastructure (regular DVNs, `CCIPDVNAdapter`, Multisig). The CCIP DVN and general DVN setup is described in [PR #1 of lz-gov-dvns-deploy](#); the `CCIPDVNAdapter` itself lives in the LayerZero v2 repository

and is expected to be deployed and configured by that library's deployer to a deterministic end state; and the multisig broadcaster/replica DVN is implemented in [PR #1 of lz-dvn-broadcaster](#).

The migration is designed to run as a single L1 spell that also relays a single Avalanche message, so that the governance-bridge reconfiguration and both OFT v2 swaps land together. The deployer is expected to have deployed and pre-configured the new relay and the new OFT v2 adapters beforehand (peers, libraries, DVNs, enforced options, fees off), with the new Avalanche adapters owned by the old relay until the spell hands them over. The migration assumes the Avalanche USDS supply is frozen and that sUSDS never bridged.

The library exposes the following entrypoints:

- `migrateAvax`: The L1 entrypoint that drives the whole migration, in the following order:
 1. **Sanity checks:** `_checkDvnOverlap`, which requires the new governance send DVN set to keep at least `MIN_DVN_OVERLAP` (4) members of the current one (so a DVN already assigned a verification job in the block stays present); and checks on the CCIP DVN adapter (its message-library and allowlist roles, its allowlist size, and that its default-admin role has been handed to the pause proxy).
 2. **Relay the Avalanche half:** dispatched through the old relay, which is still both owner and whitelisted, before the whitelist is swapped later in the same spell.
 3. **OFT swaps (USDS and sUSDS):** for each token, activate the new adapter, set its global cap, and repoint the chainlog. USDS additionally moves the frozen Avalanche backing from the old lockbox to the new one (returning the remainder so the old lockbox stays live for Solana) and severs the old Avalanche route. sUSDS needs neither, as it never bridged: there are no funds to move and its old route is already inert (rate limits 0, peer left in place), so the old adapter is simply abandoned.
 4. **Governance OApp:** write the new L1 send DVN set, then swap the gov-relay whitelist from the old relay to the new relay.
- `migrateAvaxLinked`: A thin `external` wrapper that calls `migrateAvax`. It exists to support the two deployment models: a spell that embeds the library calls `migrateAvax` directly, while a spell that links against a separately deployed copy of the library calls `migrateAvaxLinked`.
- `migrateAvaxRemote`: The Avalanche half of the migration, run as a single delegatecall by the old relay on Avalanche and exposed for relaying through the thin `LZAvaxMigrationL2Spell` wrapper, in the following order:
 1. **Remote OFTs:** activate the new remote adapters for the Ethereum route.
 2. **Governance OApp:** set the governance receiver's new receive DVN set.
 3. **Authority transfer:** move token authority (`rely` / `deny`) from the old adapters to the new adapters and from the old relay to the new relay, and transfer the delegate and ownership of the governance receiver and both new adapters to the new relay.

Remote-side sanity checks are intentionally limited, as the receive DVN set and the remote CCIP DVN adapter are assumed to have been verified off-chain.

3.3 Changelog

In [Version 2](#), the configuration checks performed by `_verifyOftConfig` were extended in response to the issue reported in [Version 1](#).

In [Version 3](#), the Avalanche migration scripts have been added. Additionally, `updateGlobalRateLimits` has been added to `LZInit`.

In [Version 4](#), `activateSsrForwarder` has been added to `LZInit` to support onboarding SSR oracle bridges, the use of the shared `CCIPDVNAdapter` has been made optional for `wireGovPeer`, and `activateOft` now receives the expected endpoint as an explicit argument.

In **Version 5**, `activateOft` additionally receives the expected OFT implementation and checks it against the proxy's own, the Avalanche migration now verifies both L1 lockboxes' aggregate rate-limit accounting type, and `USDS_OFT_IMP` and `SUSDS_OFT_IMP` are now written to chainlog.

4 Trust Model

The library has no on-chain access control; Sky governance is the sole intended caller and is fully trusted to use it correctly with well-formed parameters.

Chainlog

`LZInit` resolves the L1 governance OApp (`LZ_GOV_SENDER`) and the L1 governance relay (`LZ_GOV_RELAY`) from a chainlog contract at the hardcoded address `0xdA0Ab1e0017DEbCd72Be8599041a2aa3bA7e740F` (the Sky chainlog on Ethereum mainnet). The chainlog, and the parties able to update its entries, are part of the trusted base: Sky governance is assumed to keep these entries pointing at the correct contracts. Since **Version 3**, the Avalanche migration also writes chainlog entries, and since **Version 4** the SSR forwarder verification additionally resolves `SUSDS` and `MCD_PAUSE_PROXY` from it, so a wrong entry weakens the corresponding check rather than causing it to fail.

Deployment and privileged roles

Beyond Sky governance and the chainlog, the following parties are part of the trusted base. Those introduced with the Avalanche migration and the SSR oracle forwarder are relevant from **Version 3** and **Version 4** onwards respectively.

- **The deployer.** All contracts that the library configures are deployed and pre-configured off-chain. The library can only confirm that expected values are present; bytecode, storage and the absence of unexpected entries are assumed to have been verified by Sky governance beforehand. See `Ordering of Configuration` and `Deployment verification`.
- **LayerZero.** The `EndpointV2` contract, the message libraries and their chain-default configurations are trusted. Where a configuration is validated against `getConfig`, chain defaults are substituted field by field, so a change of the defaults by LayerZero governance changes the effective configuration. See `Sanity check`.
- **The DVN set and the executor.** Message verification relies on the configured DVNs meeting the ULN threshold, and message execution on the configured executor. From **Version 3** this includes the shared `CCIPDVNAdapter` and the multisig broadcaster and replica DVNs.
- **The `CCIPDVNAdapter` role holders.** The holders of `DEFAULT_ADMIN_ROLE`, `ALLOWLIST` and `MESSAGE_LIB_ROLE` on the shared adapter control which senders it serves and under which message library. The library asserts individual role assignments and, in `activateSsrForwarder`, grants `ALLOWLIST`, but it never enumerates the role holders or the denylist.
- **The `L2GovernanceRelay` guardian set.** The new relay adds timelock and guardian functionality; any address authorised as `bud` can cancel queued governance actions.
- **The OFT adapter pausers.** The `pausers` set on each adapter can pause the corresponding bridge.
- **The old Avalanche relay.** During the migration it is simultaneously the owner of the newly deployed contracts and the whitelisted sender, until authority is handed over within the same spell.

5 System Considerations

We leverage this section to highlight findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require a modification inside the project. Instead, they should raise awareness in order to improve the overall understanding for users and developers.

SC1 Deployment Verification

System Consideration	Version 3
For the contracts deployed ahead of the migration spell, namely the two L1 lockboxes, the two Avalanche adapters, the new <code>L2GovernanceRelay</code>, the <code>LZAvaxMigrationL2Spell</code> and the CCIP DVN adapter, special care has to be taken that all contracts have been deployed correctly, since deployment is not performed by governance directly. While some variables can be checked upon initialization by the spell, the bytecode and the remaining storage have to be verified beforehand. We therefore assume that all mappings in the deployed contracts are checked for any unwanted entries. On-chain checks can at most confirm that an expected entry is present; they cannot establish the absence of unexpected ones. This is especially crucial for the <code>pausers</code> mapping on the four new adapters, the <code>bud</code> mapping on the new <code>L2GovernanceRelay</code>, the role holders on the CCIP DVN adapter, and any <code>peers</code>, <code>feeBps</code> or rate-limit entries on the adapters beyond the single route the spell verifies, which should be empty. In the case of the new <code>L2GovernanceRelay</code>, special care also has to be taken because the spell grants it ward rights on Avalanche USDS and sUSDS and transfers ownership and delegate rights of the governance receiver and both new adapters to it, without validating it in any way. A <code>bud</code> entry under a deployer-controlled address can cancel every queued governance action. Its L1 bindings, <code>l1Eid</code>, <code>l1GovernanceRelay</code> and <code>l2Oapp</code>, are equally unvalidated, and a wrong one leaves no recovery path. The adapters are UUPS proxies, so both the proxies and the implementations behind them have to be verified beforehand, including that each proxy points to the expected implementation. Note that since <code>Version 5</code>, <code>activateOft</code> checks that each proxy points to the implementation address passed in by the caller.	

SC2 Ordering of Configuration

System Consideration	Version 1
The ordering of operations is relevant as otherwise unwanted scenarios could occur. Specifically, the setup on the new remote should be validated before finalizing the setup on mainnet and other L2s. For the governance infrastructure, the following is expected: 1. The deployer deploys the infrastructure and configures it. That includes access control (i.e. owner, delegate, peer, and immutables) as well as other configurations such as the receive library and the receive-side ULN config. 2. Governance validates the configuration. 3. Governance configures the remote on mainnet. For OFTs, the following is expected: 1. The deployer deploys the OFT and configures it. That includes access control (i.e. owner and delegate) as well as the full wiring with all other peers. Note that rate limits are configured depending on whether governance prefers to activate the new OFT itself with a cross-chain spell. 2. Governance validates the deployment.	

3. Governance wires the OFT on mainnet and sends a cross-chain spell to every other existing remote to wire the new remote accordingly. If no rate limits were set for the new remote, an activation spell is sent to the new remote.

Note that the governance-infrastructure setup should be completed before the new remote is wired on any other chain. That is to ensure that an insufficiently set up governance on the new remote does not impact OFTs on other chains. However, step 3 in both sequences can be performed within the same transaction.

6 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- **Likelihood** represents the likelihood of a finding to be triggered or exploited in practice
- **Impact** specifies the technical and business-related consequences of a finding
- **Severity** is derived based on the likelihood and the impact

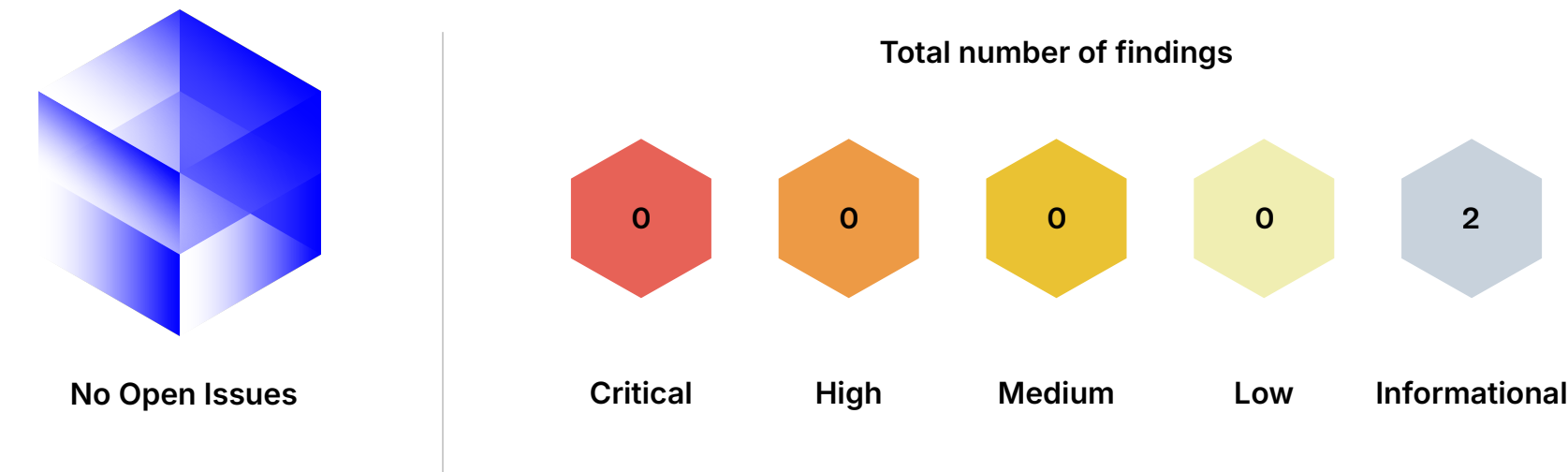
We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

7 Findings

In this section, we describe the findings identified during the course of the engagement. The findings are categorized by severity as explained in the Terminology section. Below we provide an overview of the total number of findings per severity, as well as the number of open issues.



7.1 Overview

The following table lists all identified findings along with their severity and current status. A finding is considered resolved once it has been fully corrected or the specification has been changed accordingly. Non-informational findings with any other status, including partial fixes, acknowledgements, and accepted risks, remain open.

Info	#001	Sanity Check	Code Corrected	✓
Info	#002	Chainlog OFT Implementations	Code Corrected	✓

7.2 Descriptions

#001 Sanity Check

Informational

Version 1

Code Corrected 

No amount of on-chain sanity checks can fully validate a deployment; that responsibility ultimately rests with governance's off-chain review. The checks the library does perform can nonetheless serve as an additional layer of confidence and could be made more exhaustive. Below is a non-exhaustive list:

1. `_verifyOftConfig` silently accepts the default receive library:

```
(address recvLib,) = ep.getReceiveLibrary(oft, remoteEid)
```

The above discards the second return value, a boolean indicating whether the library is the chain default. Accepting the default is potentially dangerous, as LZ governance may change it unexpectedly. The same holds for the send library (`isDefaultSendLibrary` not checked).

2. `_verifyOftConfig` accepts `UlnConfig` field-by-field defaults. The library compares against `getConfig`, which returns the merged config with chain defaults field by field, so any `UlnConfig` field stored as 0 on-chain falls through to whatever LZ has configured as the default. As a consequence, validation would pass when defaults are used on-chain and the current effective (default-substituted) values match those in `cfg`. Similarly, that applies to the executor config.
3. `_verifyOftConfig` fails to detect whether a previous receive library may still be in its grace period. `setReceiveLibrary(.., gracePeriod > 0)` leaves the old library valid in parallel with the new one until expiry, and `isValidReceiveLibrary` accepts either. `_verifyOftConfig` only reads the current library via `getReceiveLibrary` and ignores `receiveLibraryTimeout`, so it can pass while the OFT's effective receive path is the union of two libraries (with potentially different DVN sets).

Code Corrected

The code has been adjusted accordingly:

1. It is checked whether the default library is used.
2. The `UlnConfig` is now retrieved with `getAppUlnConfig` to ensure that all values set are as expected and that no merging with defaults happens (additionally, the config `confirmations` and `requiredDVNCount` are validated).
3. `EndpointV2.receiveLibraryTimeout()`'s first return value is now required to be `address(0)`, so the only other address that `isValidReceiveLibrary` could accept during a grace period is `address(0)`, which is not expected to be a meaningful library on any chain.

For point 2, this change was applied only to the `UlnConfig`. The executor config still uses `getConfig`, so the merging with defaults remains for it; Sky has confirmed this is acceptable and intended.

Furthermore, more checks have been added:

- No message inspector: `oft.msgInspector() == address(0)`
- No default fee: `oft.defaultFeeBps() == 0`
- No fee: `feeBps == 0 && !feeEnabled`

However, the only (easy to validate) variable with a setter that is not checked is `oft.preCrime()`. That is acceptable and has been confirmed to be intended by Sky.

#002 Chainlog OFT Implementations

Informational

Version 5

Code Corrected 

The chainlog is not populated with the implementation addresses behind the new USDS / sUSDS OFT proxies, as only the proxies are registered via `USDS_OFT` / `SUSDS_OFT`, whereas e.g. USDS's own implementation is registered.

Code Corrected

`USDS_OFT_IMP` / `SUSDS_OFT_IMP` are now registered, and `activate0ft` requires the passed implementation to equal the proxy's own, so a wrong value reverts instead of being recorded.

8 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.